UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

Program design: Introduction

Practical 2: Basic Classes

Release Date: 11-08-2026 at 06:00

Due Date: 14-08-2026 at 23:59

Late Deadline: 15-08-2026 at 00:59

Total Marks: 122

Read the entire specification before starting with the practical.

Contents

1 General Instructions

- ***Read the entire assignment thoroughly before you begin coding.***
- **This assignment should be completed individually.**
- **Every submission will be inspected with the help of dedicated plagiarism detection software.**
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 1 hour after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (eg. the presence/absence of certain functions or structure).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at `https://portal.cs.up.ac.za/files/departmental-guide/`.
- Unless otherwise stated, the usage of non-C++98 features or additional libraries outside of those indicated in the assignment will **not** be allowed. Some of the appropriate files that you have submit will be overwritten during marking to ensure compliance to these requirements. **Please ensure you use C++98**
- All functions should be implemented in the corresponding `cpp` file. No inline implementation in the header file apart from the provided functions.
- The usage of ChatGPT and other AI-Related software to generate submitted code is strictly forbidden and will be considered as plagiarism.
- The use of this practical, in any form, by any company/business/organisation outside the University of Pretoria is strictly forbidden. This includes, but is not limited to, the use of the practical for discussion sessions, review sessions, marketing tools, or providing certain aspects as a free service.

2 Overview

Structures and classes are the foundation of good program design and object-oriented programming (OOP). Building on your existing proficiency with structs, this practical introduces basic classes through the implementation of a simple file management system.

2.1 Mark Breakdown

The following table represents the total marks assigned to each task on FitchFork:

Task	Mark
Metadata	27
File	24
Folder	22
Getters	18
Setters	19
Testing	12
Total	122

3 Your Task:

You are required to implement all of the functions laid out in this section as well as complete the design part laid out in Section 3.5. The design component involves drawing UML Object diagrams for a given program.

Note: The functions below are written in UML notation, therefore, there is no rear `const` shown. All functions that require it are indicated as such in the provided header files.

3.1 Metadata.h

The `Metadata` class is the foundation that both `File` and `Folder` build on. It holds the properties every file and folder carries: its size, a short description, when it was created, when it was last modified, and what you're allowed to do with it (read, write, or full access). The header also defines a `Date` struct for storing and comparing calendar dates, a `Permission` enumeration (enum) for the access levels, a global `indent` function used to line up printed output neatly and a printer function. The `Date` struct and `Permission` enum live in the header so that all other files can access them.

3.1.1 struct Date

- Holds a calendar date as three integers: `day`, `month`, and `year`.
- For example, the date the 10th of August 2026 would be stored as `Date{10, 8, 2026}`.

3.1.2 enum Permission

- Describes what actions are allowed on a file or folder.
- Has three values: `read`, `write`, `full`.
- *For more information on enum: https://www.geeksforgeeks.org/cpp/enumeration-in-cpp/*

3.1.3 Members

– `size:int`

- how large the file or folder is in bytes

– `description:string`

- a short note about the file

– `dateCreated:Date`

- the date it was made

– `lastModified:Date`

- the date it was most recently changed

– `permission:Permission`

- one of the `Permission` values above

3.1.4 Functions

The functions specified in this section should be implemented in `Metadata.cpp`

3.1.4.1 isBefore(a: Date&, b: Date&): bool

- A helper function implemented for you as an inline function. Do not change it.
- It is used to check if `a` is before `b`.

3.1.4.2 indent(level: int): string

- A helper function implemented for you as an inline function. Do not change it. Refer to 3.4
- It is used by each class's `toString()` function to format the output.
- *Note: `indent()` is a global function. It does not belong to `Metadata`, it just lives in `Metadata.h`.*

3.1.4.3 + Metadata()

- Default constructor. Used when you make a file or folder with no details provided.
- The default values are:
  - `size` = 0
  - `description` = `"New file/folder"`
  - `permission` = `full`.
  - `dateCreated` = `10/8/2026`
  - `lastModified` = `10/8/2026`

3.1.4.4 + Metadata(size: int, description: const string& , dateCreated: Date)

- Parametised constructor.
- It populates Metadata as follows:
  - `size`, `description`, `dateCreated` from the arguments.
  - `lastModified` = `dateCreated`
  - `permission` = `full`

3.1.4.5 + getSize(): int

- Returns the size.

3.1.4.6 + getDescription(): const string&

- Returns the description.

3.1.4.7 + getDateCreated(): Date

- Returns the creation date.

3.1.4.8 + getDateModified(): Date

- Returns the last modified date.

3.1.4.9 + getPermission(): Permission

- Returns the permission.

3.1.4.10 + setSize(int size): bool

- Updates the size.
- Returns:
  - `false` and changes nothing if `size` < 0
  - `true` and stores the new value otherwise

3.1.4.11 + setDescription(description: const string&): void

- Changes the description to what is passed in.

3.1.4.12 + setDateModified(lastModified: Date): void

- Updates the modified date.
- Does nothing if the new date is:
  - before `dateCreated` or
  - before the current `lastModified`
- Otherwise the new date is stored.
- *Hint:* `isBefore(a, b)` *returns true if* `a` *is before* `b`

3.1.4.13 + setPermission(permission: Permission): void

- Sets the permission to `read`, `write` or `full`.

3.1.4.14 + toString(level: int): string

- A helper function implemented for you as an inline function. Do not change it. Refer to 3.4
- Returns an indented text block describing all of the metadata.
- `level` controls the indentation, so it nests inside a file or folder.

3.1.4.15 + ~Metadata()

- Destructor.
- No dynamically allocated memory, so nothing to clean up.

3.2 File.h

The `File` class represents a single file. A file has a name, an extension, a `Metadata` object describing it and an actual chunk of "data" stored as a dynamic 1D array of integers. The header also defines an `Extension` enum, which lists the file types the class understands (`txt`, `csv`, `mp3`), along with a global `extensionToString` helper function.

3.2.1 enum Extension

- Lists the file types the `File` understands: `txt`, `csv`, `mp3`.
- Used to build the full file name and to print the tag next to it.
- *For more information on enum: https://www.geeksforgeeks.org/cpp/enumeration-in-cpp/*

3.2.2 Members

– `name:string`

- the file's name without the extension (e.g. `"song"`)

– `ext:Extension`

- which `Extension` it is

– `metadata:Metadata`

- its `Metadata` object

– `dataSize:int`

- how many integers are stored in the contents

– `data:int*`

- a pointer to the array of integers, or `NULL` when the file is empty

3.2.3 Functions

The functions specified in this section should be implemented in `File.cpp`

3.2.3.1 extensionToString(ext: Extension): string

- A helper function implemented for you as an inline function. Do not change it.
- It turns the enum into readable text.
- Returns:
  - `"txt"` for `txt`
  - `"csv"` for `csv`
  - `"mp3"` for `mp3`

3.2.3.2 + File()

- Default constructor. Makes an empty file.
- The default values are:
  - `name` = `"Untitled"`
  - `ext` = `txt`
  - `metadata` = default `Metadata`
  - `dataSize` = 0
  - `data` = `NULL`

3.2.3.3 + File(name: const string&, ext: Extension, description: const string&, dateCreated: Date)

- Parametised constructor.
- It sets:
  - `name`, `ext` from the arguments
  - `metadata` from `description` and `dateCreated` with starting size 0
  - `dataSize` = 0
  - `data` = `NULL`

3.2.3.4 + File(other: const File&)

- Copy constructor.
- Copies `name`, `ext` and `metadata` from `other`.
- Calls `copyDataFrom()` to duplicate the data.

3.2.3.5 − copyDataFrom(other: const File&): void

- Private helper that performs a **deep copy** of the data array.
- What it does:
  - if `other's data` is `NULL` this file's data stays `NULL` and the function returns.
  - otherwise it copies `other's dataSize` and allocates a new array of the same size and copies every value (or sets them all to 1)

3.2.3.6 − clearFile(): void

- Private cleanup helper.
- What it does:
  - deletes the data array if it exists
  - sets `data` = `NULL`
  - resets `dataSize` = 0
  - resets `metadata`'s size = 0

3.2.3.7 − allocateFile(dataSize: int): void

- Private helper that gives the file new contents.
- What it does:
  - clears any old data
  - allocates a new array of the requested size
  - populates every data element with the value 1 (mock bits)
  - records the new `dataSize`

3.2.3.8 + writeToFile(size: int, lastModified: Date): bool

- A public way to put data into a file
- Returns `false` if:
  - the `permission` is `read`
  - `size` is 0 or negative
- Otherwise:
  - allocates `size` integers using `allocateFile()`
  - sets the `metadata` size to `size` × `sizeof(int)` bytes
  - updates the modified date
  - returns `true`

3.2.3.9 + getName(): const string&

- Returns the name (without the extension).

3.2.3.10 + getExt(): Extension

- Returns the extension.

3.2.3.11 + getMetadata(): const Metadata&

- Returns the file's metadata.

3.2.3.12 + getDataSize(): int

- Returns dataSize.

3.2.3.13 + setName(name: const string&): void

- Renames the file.

3.2.3.14 + setExt(ext: Extension): void

- Changes the file's extension.

3.2.3.15 + fullName() const: string

- Returns the name and extension with a dot to create the full name.
- Example: ‘‘`song`’’ + `mp3` returns ‘‘`song.mp3`’’.

3.2.3.16 + toString(level: int): string

- A helper function implemented for you as an inline function. Do not change it. Refer to 3.4
- Returns a printable description of the file: full name and type, metadata and contents.
- Contents line:
  - `(empty)` if the file has no data
  - otherwise the value count followed by the values
- Example (`toString(0)` on `notes.txt` after writing three values):

```
File: notes.txt  [txt]
  Metadata:
    size:        12
    description: my notes
    created:     05/08/2026
    modified:    06/08/2026
    permission:  full
  Contents: (3 values)
    1 1 1
```

3.2.3.17 + ~File()

- Deallocates the data array.

3.3 Folder.h

The `Folder` class represents a folder that can hold both files and other folders (its sub-folders), letting you build a whole tree of nested content. Like `File`, it manages its own memory, storing dynamic arrays of pointers to the files and sub-folders it owns. It also keeps a few running counts: how many files sit directly inside it, how many sub-folders it has and the total number of files nested anywhere inside it.

3.3.1 Folder

3.3.2 Members

- `name:string`
  - the folder's name
- `metadata:Metadata`
  - its `Metadata` object
- `files:File**`
  - a dynamic array of pointers to the `File`s directly inside this folder
- `subFolders:Folder**`
  - a dynamic array of pointers to the `Folder`s directly inside this folder
- `numFiles:int`
  - how many files sit directly in this folder
- `numFolders:int`
  - how many sub-folders sit directly in this folder
- `totalFiles:int`
  - the grand total of files, counting everything nested inside sub-folders too

3.3.3 Functions

The functions specified in this section should be implemented in `Folder.cpp`

3.3.3.1 + Folder()

- Default constructor. Creates an empty folder.
- The default values are:
  - `name` = `"Untitled"`
  - `metadata` = default `Metadata`
  - `files` = `NULL`, `subFolders` = `NULL`
  - `numFiles` = `numFolders` = `totalFiles` = 0

3.3.3.2 + Folder(name: const string&, description: const string&, dateCreated: Date, inputFiles: File*&, count: int)

- Parametised constructor.
- If `count` > 0 and `inputFiles` is not `NULL`:
  - it deep copies the given files
  - sets `subfolders` to `NULL`
  - adds up the files' sizes and stores it in the folder's metadata
  - `numFiles` = `totalFiles` = `count`
- If `count` is 0 (or the `inputFiles` is `NULL`) then just an empty named folder is created.

3.3.3.3 + Folder(other: const Folder&)

- Copy constructor.
- Copies `name`, `metadata` and all the tracked counts.
- Deep copies every file and every sub-folder.
- Because sub-folders are copied too, copying the top folder copies the whole folder structure beneath it.

3.3.3.4 + getName(): const string&

- Returns the folder's name.

3.3.3.5 + getMetadata(): const Metadata&

- Returns the folder's metadata.

3.3.3.6 + getNumFiles(): int

- Returns the number of files.

3.3.3.7 + getNumFolders(): int

- Returns number of folders.

3.3.3.8 + getTotalFiles(): int

- Returns total number of files.
- It includes the files in sub-folders in the total count.

3.3.3.9 + addFile(file: const File& , lastModified: Date): void

- Adds a copy of a file into the folder.
- What it does:
  - grows the files array by one
  - stores a deep copy of the file
  - `numFiles` and `totalFiles` += 1
  - adds the file's size onto the folder's metadata size
  - updates the modified date

3.3.3.10 + addFolder(folder: const Folder&, lastModified: Date): void

- Adds a copy of an entire sub-folder into this folder.
- What it does:
  - grows the sub-folders array by one
  - stores a deep copy of the sub-folder
  - `numFolders` += 1
  - `totalFiles` += the sub-folder's `totalFiles`
  - adds its size onto the metadata size
  - updates the modified date
- Example: Adding a sub-folder with 3 files leads to `numFolders` +1, `totalFiles` +3.

3.3.3.11 + toString(level: int): string

- A helper function implemented for you. Do not change it. Refer to 3.4
- Prints the folder and recursively, everything inside it.
- It shows:
  - the folder name
  - the file, sub-folder and total counts
  - the metadata
  - a "Contents" section with sub-folders first then files

- Example:

```
Folder: Music
  Files:       1
  Sub-Folders: 1
  Total files: 2
  Metadata:
    size:        24
    description: my music
    created:     05/08/2026
    modified:    06/08/2026
    permission:  full
  Contents:
    Folder: Albums
      Files:       1
      ...
    File: song.mp3  [mp3]
      ...
```

3.3.3.12 + ~Folder()

- Destructor.
- What it does:
  - deletes every file and every sub-folder it owns
  - deallocates the two pointer arrays

3.4 Note on + `toString(level:int)`

This function is purely for formatting purposes. The level is always 0 when called from its own class. For example, when `toString(0)` is called on `File`, it means you want to see the file structure from the point where that file is the root (i.e its information, metadata and data). If you call `toString(0)` on a `Folder` it means you want to see the file structure from the point where that folder is the root (i.e its information, metadata, files, sub-folders etc). `Folder`'s `toString()` will call the other classes' versions with the correct indent.

3.5 Design

For the design component of this practical, you are required to draw UML object diagrams answering the questions based on the program below. Your diagrams must follow the UML standards taught in class and each page of your `draw.io` file must contain exactly one diagram (i.e. one question per page). Remember to show other objects that are also created and show the relationships between objects.

Your `draw.io` file should be submitted to the "Practical 2 - Design" slot on the CS Portal.

```
Date jan = {2, 1, 2024};
Date feb = {28, 2, 2024};
Date mar = {5, 3, 2024};
Date apr = {25, 4, 2024};
Date may = {8, 5, 2025};
Date jun = {22, 6, 2025};
Date jul = {11, 7, 2025};
Date aug = {18, 8, 2025};
Date sep = {15, 9, 2026};
Date oct = {14, 10, 2026};
Date nov = {17, 11, 2026};
Date dec = {10, 12, 2026};

File water("Water", mp3, "Water - Tyla", jan);
File coconutWater("Coconut Water", mp3, "Coconut Water - Trim", feb);
File apt("APT", mp3, "APT - ROSE & Bruno Mars", mar);
File anxiety("Anxiety", mp3, "Anxiety - Doechii", apr);

File readme("readme", txt, "Project intro", may);
File grocery("Grocery list", txt, "Things to get month end", jun);
File todo("To-do", txt, "Things I need to do every this week", jul);
File notes("COS110 Notes", txt, "Notes I took during the UML Object Diagrams
    lecture", aug);

File config("config", csv, "Function configurations", sep);
File budget("Budgeting", csv, "Somewhere to try and manage my finances", oct);
File marks("Semester 1 Marks", csv, "A marksheet of all my modules", nov);
File exercise("ALL 121", csv, "Friday's class practise that I need to import in
    Excel", dec);

File chanel("Chanel", mp3, "Chanel - Tyla", jan);
File didItAgain("She did it again", mp3, "She did it again - Tyla (feat. Zara
    Larsson)", jan);
File push2Start("Push 2 Start", mp3, "Push 2 Start - Tyla", jan);
File truthOrDare("Truth Or Dare", mp3, "Truth Or Dare - Tyla", jan);

File popularSongsArr[5] = {water, chanel, didItAgain, push2Start, truthOrDare};
File* popularSongs = popularSongsArr;

```

```
Folder tylaDiscography("Tyla Discography", "List of Tyla's songs", feb,
    popularSongs, 5);

notes.writeToFile(5, sep);

File notesCopy(notes);
File marksCopy(marks);

Metadata notesCopyMetadata = notesCopy.getMetadata();
notesCopy.setName("COS 110 Notes - Object Diagrams");

notesCopy.writeToFile(9, oct);
marksCopy.writeToFile(8, dec);

File singlesArr[3] = {coconutWater, apt, anxiety};
File* singles = singlesArr;
Folder recentSingles("Recent Singles", "2024 singles", sep, singles, 3);

File* none = NULL;
Folder library("My Library", "Everything", sep, none, 0);
library.addFolder(tylaDiscography, oct);
library.addFolder(recentSingles, nov);
library.addFile(readme, dec);
```

These questions pertain to objects created before line 28:

1. Draw the object diagram for any `mp3 File` object.
2. Draw the object diagram for any `txt File` object.
3. Draw the object diagram for any `csv File` object.

These questions pertain to objects created before line 50:

4. Draw the object diagram for `tylaDiscography`.
5. Draw the object diagram for `notesCopy`, `notesCopyMetadata` and `marksCopy`.

This question pertains to the end of the program:

6. Draw the object diagram for `library`.

4 Memory Management

As memory management is a core part of C++ programming, each task on FitchFork will allocate approximately 10% of the marks to memory management. The following command is used:

```
valgrind --leak-check=full ./main
```

Please ensure, at all times, that your code *correctly* de-allocates *all* the memory that was allocated.

5 Testing

As testing is a vital skill that all software developers need to know and be able to perform daily, 10% of the assignment marks will be allocated to your testing skills. To do this, you will need to submit a testing main (inside the `main.cpp` file) that will be used to test an Instructor Provided solution. You may add any helper functions to the main.cpp file to aid your testing. In order to determine the coverage of your testing the gcov [1] tool, specifically the following version *gcov (Debian 8.3.0-6) 8.3.0*, will be used. The following set of commands will be used to run gcov:

```
g++ --coverage *.cpp -o main
./main
gcov -f -m -r -j ${files}
```

This will generate output which we will use to determine your testing coverage. The following coverage ratio will be used:

$$\frac{\text{number of lines executed}}{\text{number of source code lines}}$$

We will scale this ration based on class size.

The mark you will receive for the testing coverage task is determined using Table 1:

Coverage ratio range	% of testing mark
0%-5%	0%
5%-20%	20%
20%-40%	40%
40%-60%	60%
60%-80%	80%
80%-100%	100%

Table 1: Mark assignment for testing

Note the top boundary for the Coverage ratio range is not inclusive except for 100%. Also, note that only the functions stipulated in this specification will be considered to determine your testing mark. Remember that your main will be testing the Instructor Provided code and as such, it can only be assumed that the functions outlined in this specification are defined and implemented.

As you will be receiving marks for your testing main, plagiarism detection will also be performed on the testing main.

[1] For more information on gcov please see `https://gcc.gnu.org/onlinedocs/gcc/Gcov.html`

6 Implementation Details

- You must implement the functions in the header files exactly as stipulated in this specification. Failure to do so will result in compilation errors on FitchFork.
- You may only use **c++98**.
- You may use `namespace std`.
- You may only utilise the specified libraries. Failure to do so will result in compilation errors on FitchFork.
- You may only use the following libraries:
  - `<string>`
  - `<iostream>`
  - `<sstream>`
  - `<iomanip>`
  - `<fstream>`
- You are supplied with a **trivial** main demonstrating the basic functionality of this assessment. The expected output is included in the `output.txt` file.

7 Upload Checklist

The following c++ files should be in a zip archive named uXXXXXXXX.zip where XXXXXXXX is your student number:

- `Metadata.h`
- `Metadata.cpp`
- `File.h`
- `File.cpp`
- `Folder.h`
- `Folder.cpp`
- `main.cpp`
- Any textfiles used by your `main.cpp`

The files should be in the root directory of your zip file. In other words, when you open your zip file you should immediately see your files. They should not be inside another folder.

8 Submission

You need to submit your source file(s), as a single archive, on the FitchFork website (`https://ff.cs.up.ac.za/`). All methods need to be implemented (or at least stubbed) before submission. Your code should be able to be compiled with the following command:

```
g++ -std=c++98 -Werror -Wall *.cpp -o main
```

and run with the following command:

```
./main
```

Remember your `h` files will be overwritten, so ensure you do not alter the provided `h` files.
You have 10 submissions, and your best mark will be your final mark. Upload your archive to the Practical 2 slot on the FitchFork website. If you submit after the deadline but before the late deadline, a 20% mark deduction will be applied. **No submissions after the late deadline will be accepted!**